



**UNIVERSIDAD DE EL SALVADOR EN LÍNEA  
FACULTAD DE INGENIERÍA Y ARQUITECTURA  
ESCUELA DE INGENIERÍA DE SISTEMAS  
INFORMÁTICOS  
EDUCACION A DISTANCIA  
SISTEMAS EMBEBIDOS I**



## **Guía de laboratorio No. 3 Introducción a Tinkercad.**

### **OBJETIVO GENERAL.**

- Utilizar el entorno de desarrollo de circuitos en línea de Autodesk Tinkercad.

### **OBJETIVO ESPECÍFICOS.**

- Usar Tinkercad para la elaboración y simulación de circuitos electrónicos.
- Usar Tinkercad para el desarrollo de sketch para sistemas embebidos con microcontrolador basado en Arduino.
- Usar un entorno de desarrollo en línea para verificar el funcionamiento de sketch y circuitos en conjunto.

### **EQUIPO, MATERIALES y HERRAMIENTAS.**

- Computadora Personal con Windows (o Linux).
- Conexión a Internet y navegador web para dar seguimiento a la actividad.

### **INTRODUCCIÓN.**

#### **Tinkercad.**

Es una aplicación de tipo en línea utilizada para el diseño e impresión 3D, desarrollada por Autodesk. La aplicación permite realizar simulaciones en tiempo real, programación de dispositivos tales como Arduino en forma virtual. La programación puede llevarse a cabo en forma de texto o bloques (de forma similar a AppInventor o Scratch), también podemos exportar el circuito para realizar posteriormente un diagrama de pistas (PCB).

#### **Diodos LED<sup>1</sup>.**

Es un tipo de diodo que emite luz al polarizarse en directa, en caso de no alcanzarse dichas condiciones o polarizarse en inversa el diodo se mantiene apagado. Los circuitos digitales hacen uso de ellos para verificar el accionamiento de puertos de salida:

- La luz en sistemas de lógica positiva indica que la salida tiene un valor lógico alto (1 lógico) y el diodo apagado indica un valor lógico bajo (0 lógico).
- La luz en sistemas de lógica negativa indica que la salida tiene un valor lógico bajo (0 lógico) y el diodo apagado indica un valor lógico alto (1 lógico).

---

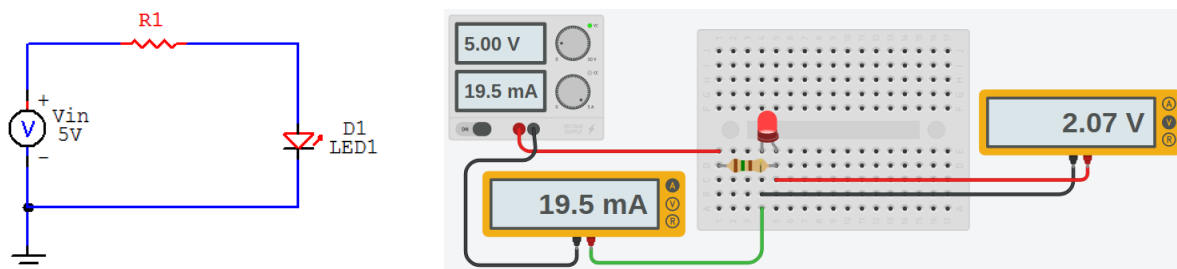
<sup>1</sup> LED: Light-Emitting Diode por su acrónimo en Inglés.

Dado que los sistemas embebidos (microcontroladores, FPGA, etc.), o sistemas integrados (SoC) funcionan en ciertas aplicaciones con lógica digital, se utilizarán leds o displays para evidenciar la respuesta del circuito digital programado. Es por ello que a continuación se muestran algunos valores de prueba<sup>2</sup> a la que se sometieron algunos diodos led:

| Material | Color    | Voltaje de polarización (Vled en V) |
|----------|----------|-------------------------------------|
| GaAs     | Morado   | 1.2                                 |
| GaAsP    | Rojo     | 1.8                                 |
| GaAsP    | Naranja  | 2.0                                 |
| GaAsP:N  | Amarillo | 2.2                                 |
| AlGaP    | Verde    | 3.5                                 |
| SiC      | Azul     | 3.6                                 |
| GaInN    | Blanco   | 4.0                                 |

**Tabla 1:** Valores de voltaje de polarización de diodos led con corriente de prueba de 20mA.

Se suele representar la salida de un microcontrolador como una fuente de tensión para realizar el análisis de voltaje de Kirchhoff para encontrar el valor óptimo de resistencia a la salida del puerto digital que regulará la corriente que entreguemos al led.



**Figura 1:** Circuito esquemático y pictórico de conexiones para el control de encendido de un led.

$$-V_{in} + V_R + V_D = 0$$

Sustituyendo el voltaje de activación del diodo led en directa, el de la fuente de alimentación por su magnitud y utilizando la ley de Ohm para la resistencia tenemos:

$$-(5) + (I_D \cdot R) + (2.07) = 0$$

$$R = \frac{5 - 2.07}{I_D} = \frac{5 - 2.07}{19.5 \text{ mA}} = 150.2564\Omega$$

### Declaración de variables<sup>3</sup> enteras (int).

Se usa para el almacenamiento de números. En las tarjetas Arduino Uno (y otras basadas en ATmega), un int almacena un valor de 16 bits (2 bytes), un rango de valores de -32,768 ( $-2^{15}$ ) a 32,767 ( $(2^{15}) - 1$ ). En las placas basadas en Arduino Due y SAMD (como MKR1000 y Zero), un int almacena un valor de 32 bits (4 bytes). Un rango de valores -2,147,483,648 ( $-2^{31}$ ) a 2,147,483,647 ( $(2^{31}) - 1$ ). Int almacena números negativos por medio

<sup>2</sup> Valores tomados de: <https://www.mecatronicalatam.com/es/tutoriales/electronica/componentes-electronicos/diodo/diodo-led/>

<sup>3</sup>La sintaxis para el desarrollo de sketch de Arduino puede ser encontrada en: <https://www.arduino.cc/reference/en/>

de complemento A2. El bit más alto, es denominado bit de signo (señala si el número es negativo). El resto de los bits se invierten y se agrega 1. Ejemplos:

```
int ledPin = 13; //Sintaxis: int nombre_variable = valor_inicial_de_la_variable;
int Retardo = 1000;
```

### Declaración de Arreglos (matrices y vectores).

Un arreglo es una colección de valores al interior de una variable a los que se accede con un número de índice. Puede declarar un arreglo sin inicializarlo como en **myInts**, lo que indica que no es necesario colocar los valores cuando se declara el arreglo. En **myPins** se declara un arreglo sin elegir explícitamente un tamaño, el compilador cuenta los elementos y crea un arreglo del tamaño apropiado. Finalmente, puede inicializar y dimensionar su arreglo, como en **mySensVals**. Todos los métodos a continuación son formas válidas para crear (declarar) arreglos.

Ejemplos:

```
int myInts[6];
int myPins[] = {2, 4, 8, 3, 6};
int mySensVals[6] = {2, 4, -8, 3, 2};
char message[6] = "hello";
int entradas[] = {2, 3, 4, 5}; //Entradas de DIP switch
int salidas[] = {6, 7, 8, 9, 10, 11, 12, 13}; //Salidas a display
bool leds[4][8]{ //Arreglo con estados de leds de display{a,b,c,d,e,f,g,pd}
    {1,1,1,1,1,1,0,0}, //0
    {0,1,1,0,0,0,0,0}, //1
    {1,1,0,1,1,0,1,0}, //2
    {1,1,1,1,0,0,1,0}}; //3
```

Tenga en cuenta que al declarar un arreglo de tipo char, se requiere un elemento más que su inicialización, para mantener el carácter nulo requerido. Los arreglos están indexados a partir de cero, el primer elemento de la matriz está en el índice 0. También significa que, en una matriz con diez elementos, el índice nueve es el último elemento. Ejemplos:

```
int myArray[10] = {9,3,2,4,3,2,7,8,9,11}; //mySensVals[0] == 2
//mySensVals[6] == 4
//myArray[9] contiene el valor 11
//myArray[10] es invalido y contiene información aleatoria
```

### Funciones iniciales.

Cada vez que se inicia un nuevo Sketch (código para tarjeta Arduino), se crean dos funciones la primera la función **void setup** se ejecuta una sola vez y sirve por ejemplo para realizar la configuración de pines físicos de la tarjeta indicando en ella de los pines a utilizar cuáles serán entradas o salidas. La segunda **void loop** debe contener un código que se repite constantemente (las luces de un semáforo, por ejemplo) o la aplicación principal que deseamos desarrollar (control de un proceso de lavado de ropa). Ejemplo:

```
void setup() {
    // ponga su codigo de configuracion aqui, se corre una vez:
}

void loop() {
    // ponga su codigo principal aqui, se corre repetidamente:
}
```

### Declaración de puertos digitales como entradas o salidas.

Se utiliza la función pinMode para definir la dirección de datos de un pin digital. Se suele colocar esta función al interior de la función void setup. Ejemplo:

```
int pin = 3;
```

```
pinMode(LED_BUILTIN, OUTPUT); //Sintaxis: pinMode(Puerto, dirección);
pinMode(13, OUTPUT);
pinMode(2, INPUT);
pinMode(pin, INPUT_PULLUP);
```

### Escritura de puertos digitales (salidas).

Escribe un valor HIGH o LOW a un pin digital. Si el pin ha sido configurado como OUTPUT con pinMode(), su voltaje se ajustará al valor correspondiente: 5V (o 3.3V en placas de 3.3V) para HIGH, 0V (tierra) para LOW. Si el pin está configurado como INPUT, digitalWrite() habilitará (HIGH) o deshabilitará (LOW) el pullup interno del pin de entrada. Se recomienda establecer el pinMode() a INPUT\_PULLUP para habilitar la resistencia interna de pull-up. Vea el tutorial de Pines Digitales para más información.

Si no establece el pinMode() a OUTPUT, y conecta un LED a un pin, al llamar a digitalWrite(HIGH), el LED puede aparecer atenuado. Sin establecer explícitamente pinMode(), digitalWrite() habrá habilitado la resistencia interna de pull-up, que actúa como una gran resistencia limitadora de corriente. Ejemplo:

```
int pin = 3;
digitalWrite(LED_BUILTIN, HIGH); //Sintaxis: digitalWrite(pin, valor_lógico_de_salida);
digitalWrite(13, HIGH);
digitalWrite(2, LOW);
digitalWrite(pin, HIGH);
```

### Lectura de puertos digitales (entradas).

Lee el valor del pin digital, el resultado será HIGH o LOW. Ejemplo:

```
int pin = 3;
int valor;
int x = 0;
valor = digitalRead(13); //Sintaxis: valor_lógico_de_entrada = digitalRead(pin);
x = digitalRead(pin);
```

### Bucle condicional if (else):

La instrucción if comprueba una condición y ejecuta la instrucción posterior o el conjunto de sentencias si la condición es verdadera. Si se ejecuta la instrucción if else, se ejecutarán las sentencias del bloque if si la condición se cumple sino se ejecutarán las sentencias del bloque else. Sintaxis:

```
*****
1er caso:
*****
if (condición) {
    //sentencias
}
*****
2do caso:
*****
if (condición_1) {
    //Sentencias A
}
else if (condición_2) {
    //Sentencias B
}
else {
    //Sentencias C
}
*****
3er caso:
*****
if (condición_1) {
    //Sentencias A
```

```

}
else if (condición_2) {
    //Sentencias B
}
else {
    //Sentencias C
}

```

### Ejemplo de utilización:

```

if (x > 120){
    digitalWrite(LEDpin1, HIGH);
    digitalWrite(LEDpin2, HIGH);
}

```

### Bucle repetitivo –for:

La instrucción for se usa para repetir un bloque de instrucciones encerrado entre llaves. Un contador de incremento se usa generalmente para incrementar y terminar el ciclo. La sentencia for es útil para cualquier operación repetitiva, y se usa a menudo en combinación con arreglos para operar en colecciones de datos / pines. La inicialización ocurre primero y exactamente una vez. Cada vez que pasa el ciclo, la condición se prueba; si es verdadero, el bloque de instrucciones y el incremento se ejecuta, entonces la condición se prueba nuevamente. Cuando la condición se vuelve falsa, el ciclo finaliza. Sintaxis:

```

for (inicialización; condición; incremento) {
    //sentencias
}

```

### Ejemplo:

```

for (int i=0; i <= 255; i++){
    analogWrite(PWMPin, i);
    delay(10);
}

```

## PROCEDIMIENTO.

### Parte I: Simulación del sistema en Tinkercad.

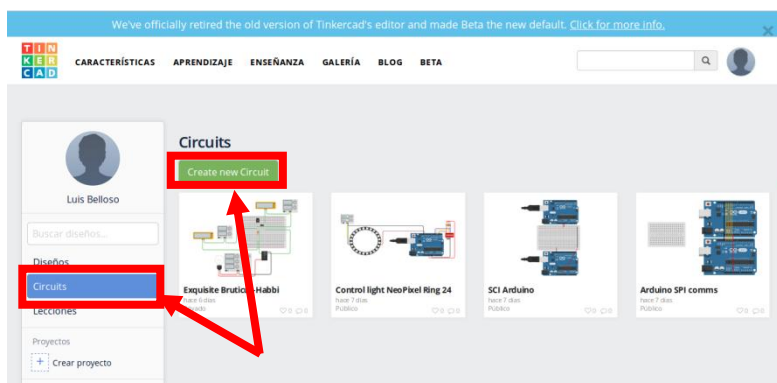
1. Encienda la computadora.
2. Abra el **navegador web** (Mozilla Firefox, Google Chrome).
3. Escriba en la barra de direcciones: **<https://www.tinkercad.com>**
4. Dé clic izquierdo en el botón **Registrarse**, ver figura 2.



**Figura 2:** Página web de TINKERCAD.

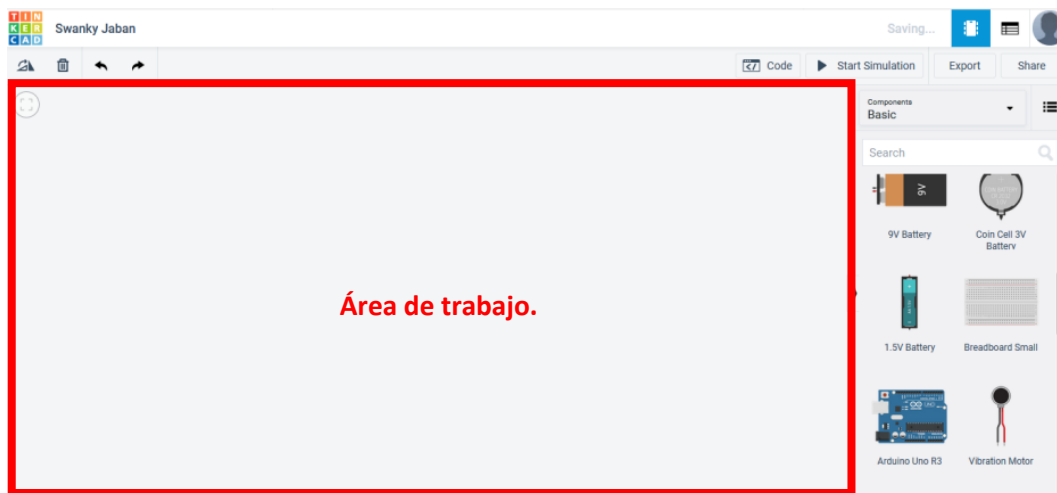
- En la página que se ha cargado, ingrese sus datos de **país de residencia**, **fecha de nacimiento** y dé clic en el botón **siguiente**, ver figura 3.
- Luego ingrese una cuenta de correo personal y una contraseña para el sitio de TINKERCAD, ellos sugieren algunos parámetros para el fortalecimiento de la misma, ver figura 3.
- Luego dé clic en el botón **Crear Cuenta**, ver figura 3.
- Cuando cargue la página principal dé clic izquierdo en **Circuits**, luego en **Create New Circuit**, ver figura 4.
- Si no se carga la ventana de la figura 4, dé clic izquierdo en el icono TINKERCAD de la esquina superior izquierda y repita el paso anterior.

**Figura 3:** Creando cuenta en TINKERCAD.



**Figura 4:** Creando circuito en TINKERCAD.

- Para agregar los componentes de la sección derecha **Components Basic**, utilice la barra de desplazamiento lateral y baje hasta lograr ver **Breadboard Small**.
- Dé clic izquierdo sobre el componente a agregar y luego un clic izquierdo sobre el área de trabajo (izquierda de Components Basic y debajo de los botones **Rotate**, **Delete**, **Undo** e **icon-rodo**), ver figura 5.



**Figura 5:** Agregando componentes en TINKERCAD.

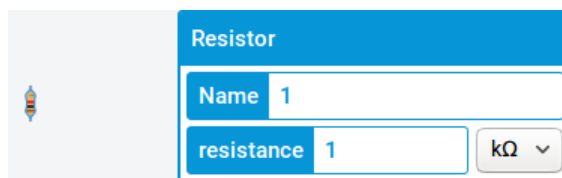
12. De la misma forma que el paso anterior, agregue los componentes:

| Nº | Cantidad | Componente       | Ubicación          |
|----|----------|------------------|--------------------|
| 1  | 1        | Arduino Uno R3   | Components > Basic |
| 2  | 1        | Led              | Components > Basic |
| 3  | 1        | Resistor de 1 kΩ | Components > Basic |

**Tabla 2:** Ubicación de los componentes a utilizar en TINKERCAD.

13. Dé clic izquierdo sobre el resistor, se desplegará el cuadro de diálogo que le permitirá modificar el valor y nombre del componente.

14. Asígnele el valor de 150Ω.



**Figura 6:** Modificando valores de componentes en TINKERCAD.

15. Coloque los componentes tal como se sugiere en la figura 7.

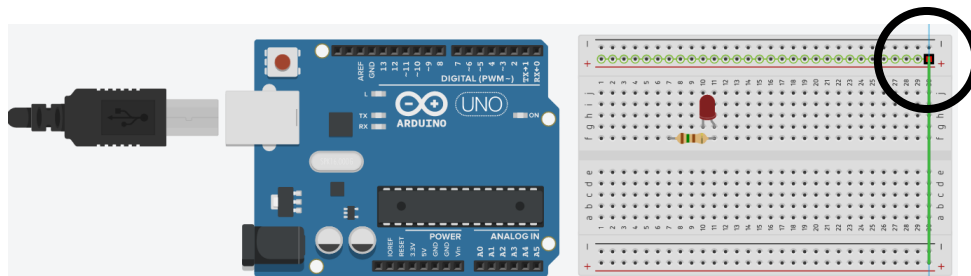
#### Notas:

- Si necesita mover un componente, dé clic izquierdo sobre él y arrástrelo hasta la posición donde lo necesita mover.
- Moviendo el scroll del mouse puede ajustar el acercamiento (zoom) del circuito a su elección.
- Si la barra de componentes desaparece, dé clic izquierdo sobre el botón <, al lado derecho de la pantalla.
- Colocando el puntero del mouse sobre los pines del componente, despliega texto que define el nombre del pin señalado.



**Figura 7:** Cableando los componentes en TINKERCAD.

16. Se comenzará a cablear la parte de alimentación entre los extremos superior e inferior de la breadboard.
17. Ubique el puntero del mouse sobre un punto de la breadboard donde desea colocar un cable, ver figura 7.
18. Dé un clic izquierdo sobre el punto de interés (inicio).
19. Mueva el mouse hasta la posición donde desee finalizar la conexión del cable y dé clic izquierdo sobre ese punto.
20. Aparecerá un cuadro de diálogo que le permitirá cambiar el color al cable.

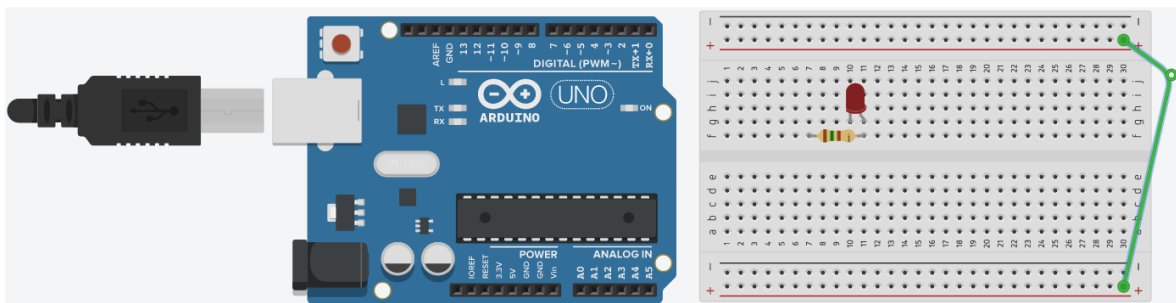


**Figura 8:** Cableando los componentes en TINKERCAD.



**Figura 9:** Definiendo el color del conductor en TINKERCAD.

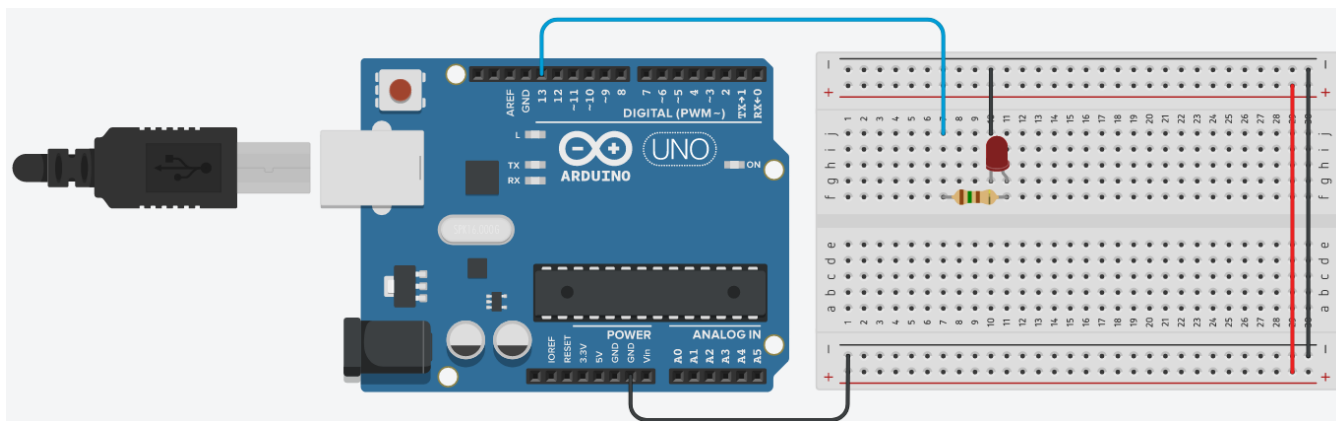
21. Al dar doble clic izquierdo sobre una sección del conductor, aparecerán puntos de anclaje sobre el punto donde se ha hecho doble clic izquierdo, que permiten modificar la trayectoria del conductor, ver figura 10.



**Figura 10:** Trayectoria del cableado definida por el usuario.



22. A mayor cantidad de veces que haga el procedimiento anterior tendrá más posibilidades de darle al conductor una trayectoria preferida.
23. Si desea eliminar un punto de anclaje selecciónelo y presione el botón suprimir.
24. Otra forma de realizar el cableado es ir dando clic izquierdo, sobre el área de trabajo definiendo la trayectoria deseada.
25. Realice las conexiones del circuito de tal manera que se vea como en la figura 11.



**Figura 11:** Circuito propuesto para el código 1.

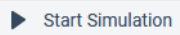
26. Dé clic izquierdo en el botón del **Code editor**,  podrá ver que por defecto tiene pre-guardado el código del ejemplo **Blink**. El cuál permite encender intermitentemente un led (encendido 1 segundo, apagado 1 segundo).
27. Agregue una variable de tipo entero llamada Retardo y asígnele el valor de 500, cambie el valor dentro de la función delay(1000) por delay(Retardo). De esta forma podrá cambiar el tiempo de intermitencia del led cambiando solo el valor de la variable Retardo.

```
int Retardo = 500;

void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(Retardo);           // Wait for 500 millisecond(s)
  digitalWrite(13, LOW);
  delay(Retardo);           // Wait for 500 millisecond(s)
}
```

**Código 1:** Modificación del Sketch Blink.ino de la biblioteca de Arduino.

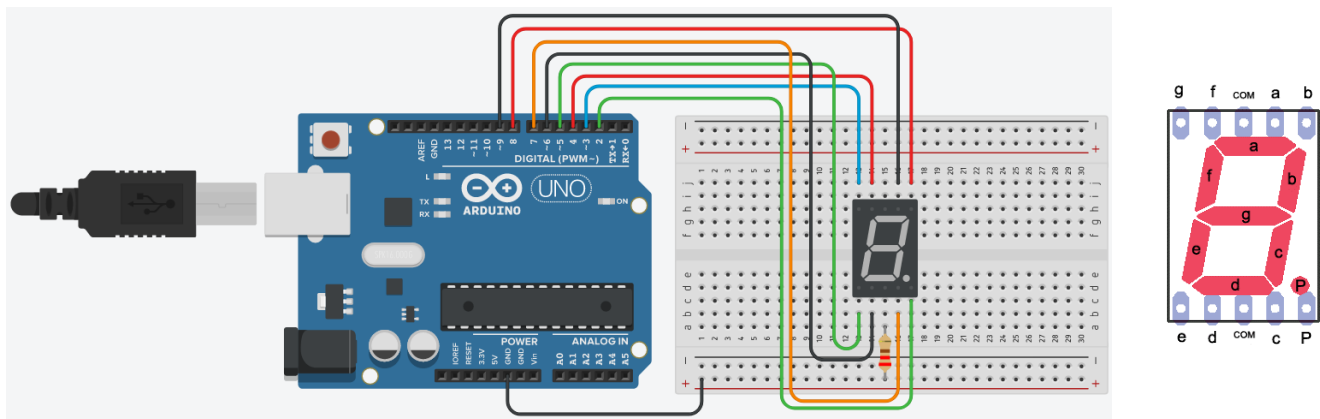
28. Dé clic izquierdo en el botón .
29. Verifique el funcionamiento del circuito y código 1.
30. Dé clic izquierdo sobre el botón **Stop Simulation**.

31. Modifique el valor de la variable **retardo** de **500** a **250**.
  32. Vuelva a correr la simulación ¿Qué efecto tiene sobre todo el código el cambio del valor de la variable Retardo?
- 
33. Al dar clic en el icono **Component List (Lista de componentes)**, le permitirá ver una lista de los componentes utilizados para realizar su aplicación. Al dar clic izquierdo en **Download CSV**, le permitirá descargar un archivo que puede abrir en un editor de hojas de cálculo.



**Figura 12:** Lista de componentes en TINKERCAD.

34. Dé clic izquierdo en el botón **Circuit View** (el botón se encuentra a la izquierda del botón Component List) para volver a la vista del circuito y código.
35. Dé clic en el icono de TINKERCAD de la esquina superior izquierda y **crea un nuevo circuito**, tal como el propuesto en la figura 13.



**Figura 13:** Utilizando display de **cátodo común** en TINKERCAD.

36. Agregue el código propuesto a continuación:

```
void setup(){
  pinMode(9, OUTPUT); //Segmento A
  pinMode(8, OUTPUT); //Segmento B
  pinMode(7, OUTPUT); //Segmento C
}
```

```

pinMode(6, OUTPUT); //Segmento D
pinMode(5, OUTPUT); //Segmento E
pinMode(4, OUTPUT); //Segmento F
pinMode(3, OUTPUT); //Segmento G
pinMode(2, OUTPUT); //Segmento PD
}

void loop(){
  digitalWrite(9, HIGH); //Valores para encender el número 0
  digitalWrite(8, HIGH);
  digitalWrite(7, HIGH);
  digitalWrite(6, HIGH);
  digitalWrite(5, HIGH);
  digitalWrite(4, HIGH);
  digitalWrite(3, LOW);
  digitalWrite(2, LOW);
}

```

**Código 2:** Parte del Sketch del decodificador de binario (4 bits) a siete segmentos, display de **cátodo común**.

**Nota:**

- Aunque el código anterior es funcional, es ineficiente porque repite varias veces una misma función, tanto para configurar el pin como para asignarle un valor lógico a la salida.
- Para códigos de este tipo se recomienda el uso de funciones repetitivas como for y el uso de arreglos.

**Parte II: Uso de vectores y bucle for.**

37. Modifique el código 2, por el sugerido en el código 3.

```

int i=0;
int Salidas[8] = {9,8,7,6,5,4,3,2}; //Segmento {A,B,C,D,E,F,G,H,PD}
bool Matriz[8] = {1,1,1,1,1,1,0,0}; //Valores para formar número 0

void setup(){
  for(i=0; i<8; i++){
    pinMode(Salidas[i], OUTPUT); //pinMode(PIN, Direccion);
  }
}

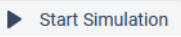
void loop(){
  for(i=0; i<8; i++){
    digitalWrite(Salidas[i], Matriz[i]); //pinMode(PIN, Valor);
  }
}

```

**Código 3:** Parte del Sketch del decodificador de binario (4 bits) a siete segmentos, display de **cátodo común**.

**Nota:**

- Como podrá observar se declararon dos vectores, uno (Salidas) para definir los pines a utilizar de la tarjeta Arduino Uno y el otro (Matriz) con los valores lógicos que deben asignarse a cada led del display para formar el número 0.

38. Dé clic izquierdo en el botón .

39. Verifique el funcionamiento del circuito y código 3. El cuál debe realizar la misma función que el código 2.

40. Dé clic izquierdo sobre el botón **Stop Simulation**.

### Parte III: Uso de bucle if.

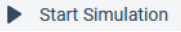
41. Modifique el código 3 por el proporcionado en el código 4.

```
int i=0;
int j=0;
int Retardo = 2000;
int Salidas[8] = {9,8,7,6,5,4,3,2};           //Segmento {A,B,C,D,E,F,G,H,PD}
bool Matriz[4][8] = {{1,1,1,1,1,1,0,0},      //Cero: 0xFC (1111 1100)
                    {0,1,1,0,0,0,0,0},      //Uno:  0x60 (0110 0000)
                    {1,1,0,1,1,0,1,0},      //Dos:  0xDA (1101 1010)
                    {1,1,1,1,0,0,1,0}};      //Tres: 0xF2 (1111 0010)

void setup(){
  for(i=0; i<8; i++){
    pinMode(Salidas[i], OUTPUT);             //pinMode(PIN, Direccion);
  }
}

void loop(){
  for(i=0; i<8; i++){
    digitalWrite(Salidas[i], Matriz[j][i]); //digitalWrite(PIN, Valor_lógico);
  }
  delay(Retardo);
  j++;
  if(j>3){
    j=0;
  }
}
```

**Código 4:** Parte del Sketch del decodificador de binario (4 bits) a siete segmentos, display de **cátodo común**.

42. Dé clic izquierdo en el botón .

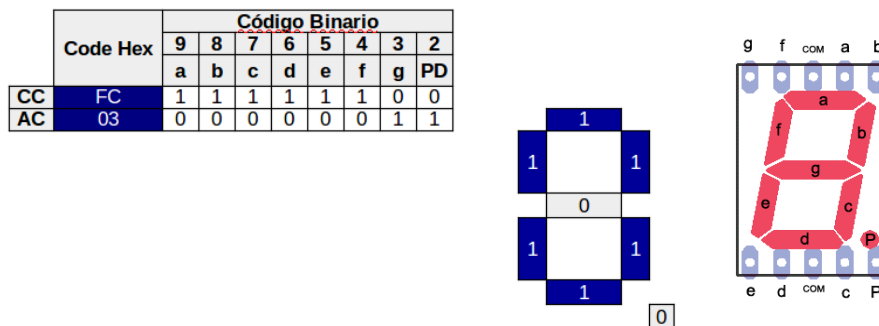
43. Verifique el funcionamiento del circuito y código 4.

44. Dé clic izquierdo sobre el botón **Stop Simulation**.

#### Nota:

- Como podrá observar solo se escriben los primeros 4 números de la tabla 3, hay que realizar modificaciones para lograr ver todos los valores sugeridos.
- Se recomienda para este tipo de ejercicios crear tablas que faciliten la escritura de los valores lógicos a asignar en el display, ver tabla 3.
- Puede utilizar la función CONCATENAR de Excel para facilitarse el trabajo de codificación con el archivo proporcionado por el docente y la figura 14, para interpretar el uso del mismo.

=CONCATENAR("{",U16,"","V16","",W16,"",X16,"",Y16,"",Z16,"",AA16,"",AB16,""},")

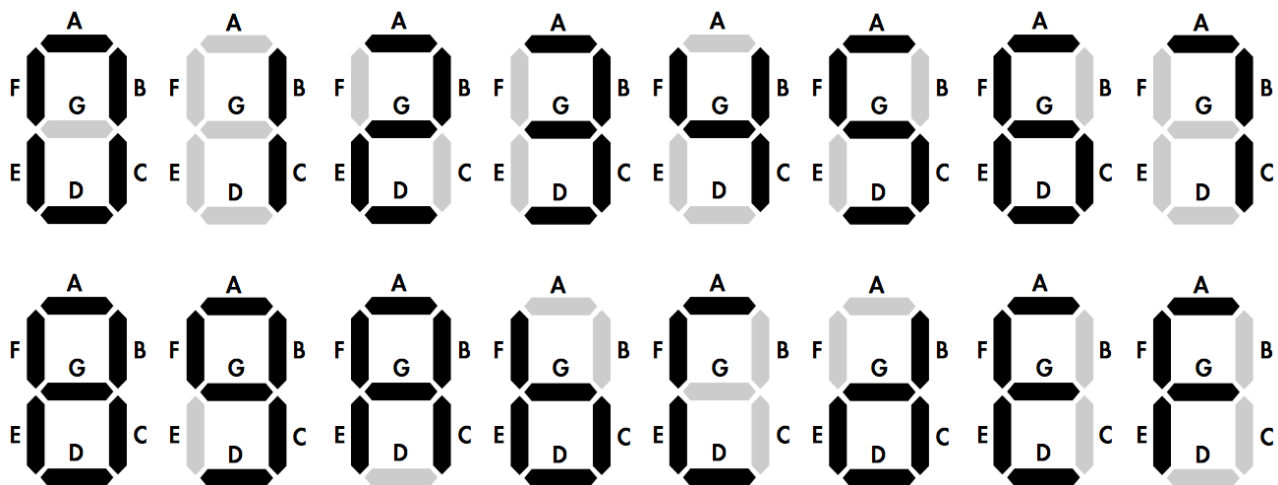


**Figura 14:** Forma de asignar los valores lógicos a cada segmento en correspondencia con los pines 9 al 2.

|      | Matriz[i] |   |   |   |   |   |   |    | Display   |   |
|------|-----------|---|---|---|---|---|---|----|-----------|---|
|      | 9         | 8 | 7 | 6 | 5 | 4 | 3 | 2  | Hex_Value |   |
|      | a         | b | c | d | e | f | g | DP |           |   |
| 0000 | 1         | 1 | 1 | 1 | 1 | 1 | 0 | 0  | FC        | 0 |
| 0001 | 0         | 1 | 1 | 0 | 0 | 0 | 0 | 0  | 60        | 1 |
| 0010 | 1         | 1 | 0 | 1 | 1 | 0 | 1 | 0  | DA        | 2 |
| 0011 | 1         | 1 | 1 | 1 | 0 | 0 | 1 | 0  | F2        | 3 |
| 0100 |           |   |   |   |   |   |   | 0  |           | 4 |
| 0101 |           |   |   |   |   |   |   | 0  |           | 5 |
| 0110 |           |   |   |   |   |   |   | 0  |           | 6 |
| 0111 |           |   |   |   |   |   |   | 0  |           | 7 |
| 1000 |           |   |   |   |   |   |   | 0  |           | 8 |
| 1001 |           |   |   |   |   |   |   | 0  |           | 9 |
| 1010 |           |   |   |   |   |   |   | 0  |           | A |
| 1011 |           |   |   |   |   |   |   | 0  |           | b |
| 1100 |           |   |   |   |   |   |   | 0  |           | c |
| 1101 |           |   |   |   |   |   |   | 0  |           | d |
| 1110 |           |   |   |   |   |   |   | 0  |           | e |
| 1111 |           |   |   |   |   |   |   | 0  |           | f |

**Tabla 3:** Valores a escribir en el display de siete segmentos (tarjeta I/O).

45. Modifique el código 4 para obtener todos los valores de la tabla 3, cambiando cada valor cada 2 segundos.

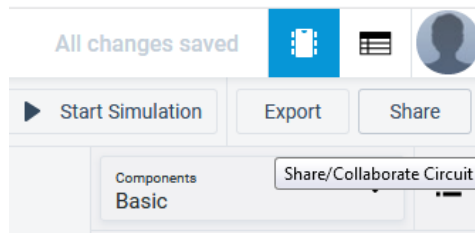


**Figura 15:** Datos hexadecimales a mostrar con la modificación del código 4.

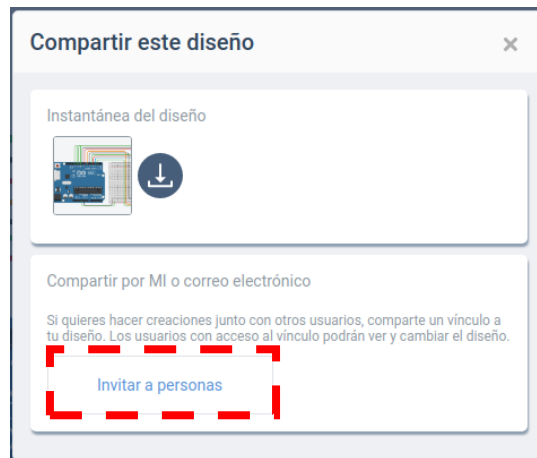
#### Parte IV: Compartir archivos de Tinkercad.

46. Dé clic izquierdo en el botón **Share/Collaborate Circuit** de la esquina superior derecha.

47. Les aparecerá una ventana en la que debe presionar el botón **Invite people (Invitar personas)**.

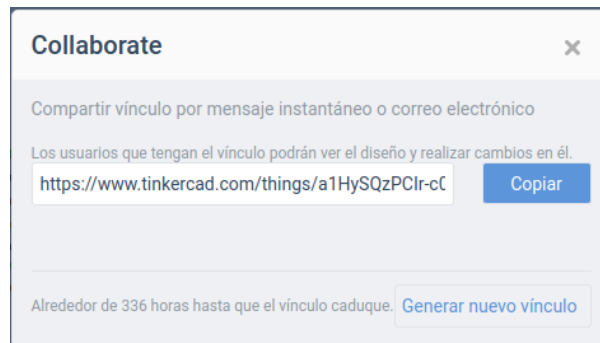


**Figura 16:** Botón para compartir trabajos de Tinkercad.



**Figura 17:** Invitar personas para compartir trabajos de Tinkercad.

48. En la nueva ventana presione el botón **Copy (Copiar)**, ver figura 18.
49. Abra su cuenta de correo, red social predilecta y pegue el contenido del portapapeles sobre un mensaje de correo electrónico o publicación de red social y envíe al destinatario de su elección.

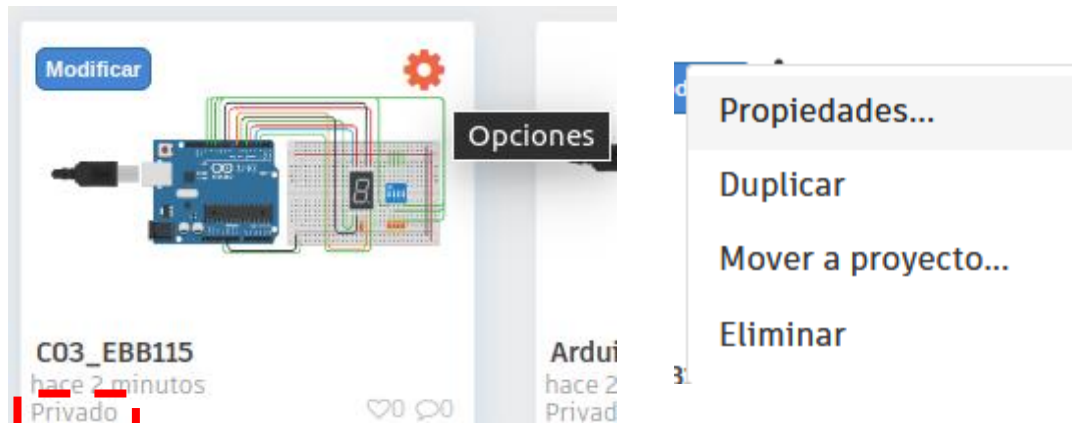


**Figura 18:** Generación de enlaces para compartir trabajos de Tinkercad.

Otra forma de compartir los archivos es:

50. En la pestaña del navegador donde tiene el Tinkercad, dé clic izquierdo en el icono de TINKERCAD.
51. Cuando cargue la página principal dé clic izquierdo en **Circuits**, obtendrá la vista de todos sus circuitos desarrollados.
52. Al colocar el mouse encima de un circuito, aparecen dos botones: **Modificar** y un engrane (**Opciones**), dé clic izquierdo sobre este último.

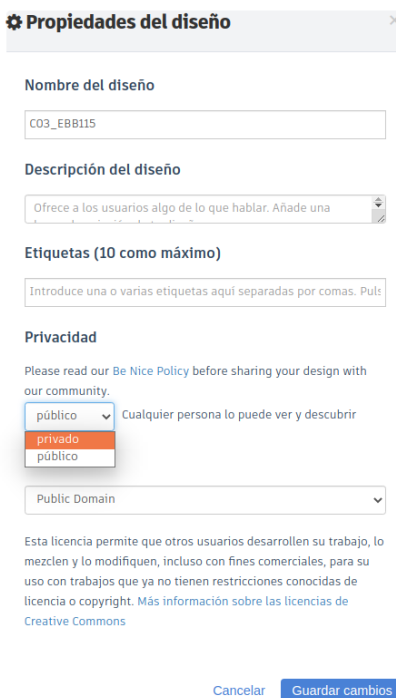
53. Seleccione la opción **Propiedades...**



**Figura 19:** Generación de enlaces para compartir trabajos de Tinkercad.

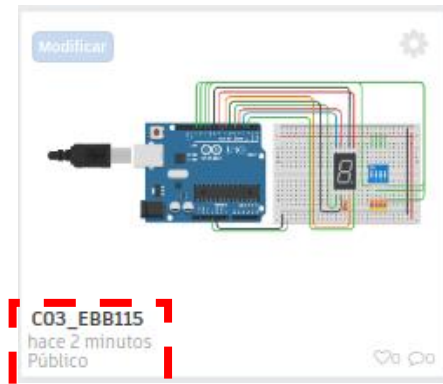
54. En la ventana desplegada seleccione la opción **público** para el archivo, con lo cual cualquier persona en Internet podrá encontrar su archivo por medio del nombre o enlace.

55. Desplácese sobre la ventana hasta la parte inferior y dé clic izquierdo en el botón **Guardar cambios**.



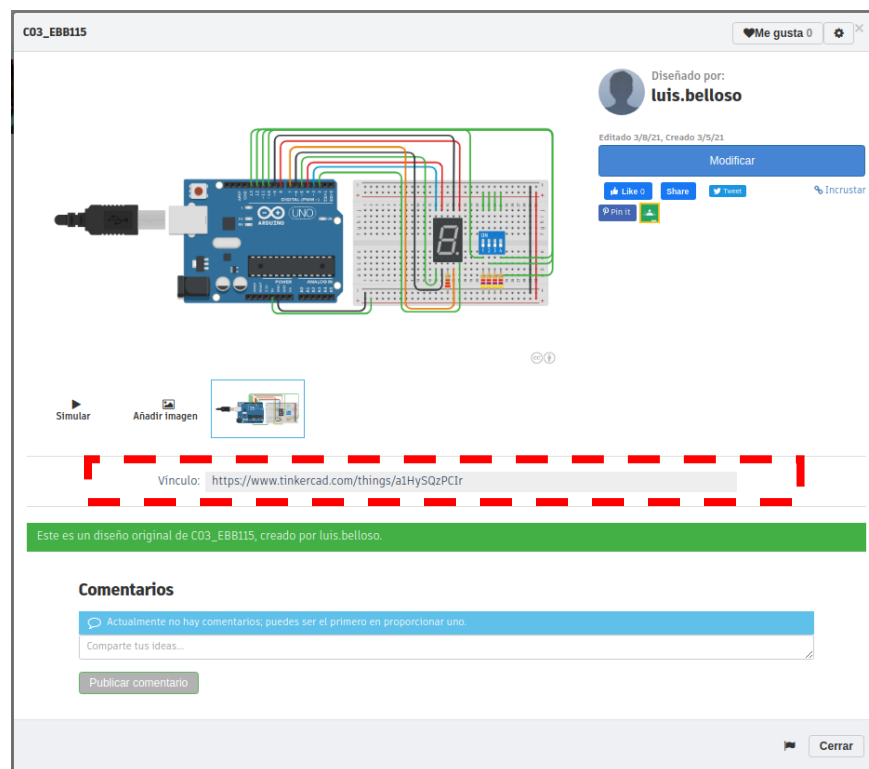
**Figura 20:** Generación de enlaces para compartir trabajos de Tinkercad.

56. Notará que el archivo ahora es de uso público, dé clic izquierdo al centro de la imagen del archivo en el navegador web.



**Figura 21:** Generación de enlaces para compartir trabajos de Tinkercad.

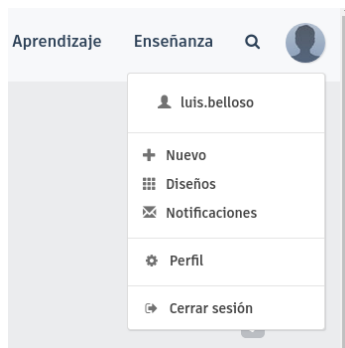
57. Desplácese a la parte inferior hasta que encuentre la dirección del vínculo a ser compartido.
58. Copiar la dirección web del archivo y comparta el archivo de Tinkercad tal como se sugirió en el paso 49 de esta guía.
59. El archivo puede volverse privado si usted así lo desea, pero hará que el enlace desaparezca de la ventana mostrada en la figura 22.



**Figura 22:** Generación de enlaces para compartir trabajos de Tinkercad.

60. Cierre la ventana de la figura 22. Puede dar clic en el botón cerrar o sobre la x de la esquina superior derecha.
61. Se cargará su perfil de TINKERCAD, dé clic izquierdo en la esquina superior derecha sobre la imagen de su perfil, luego dé clic izquierdo en **Cerrar sesión**.





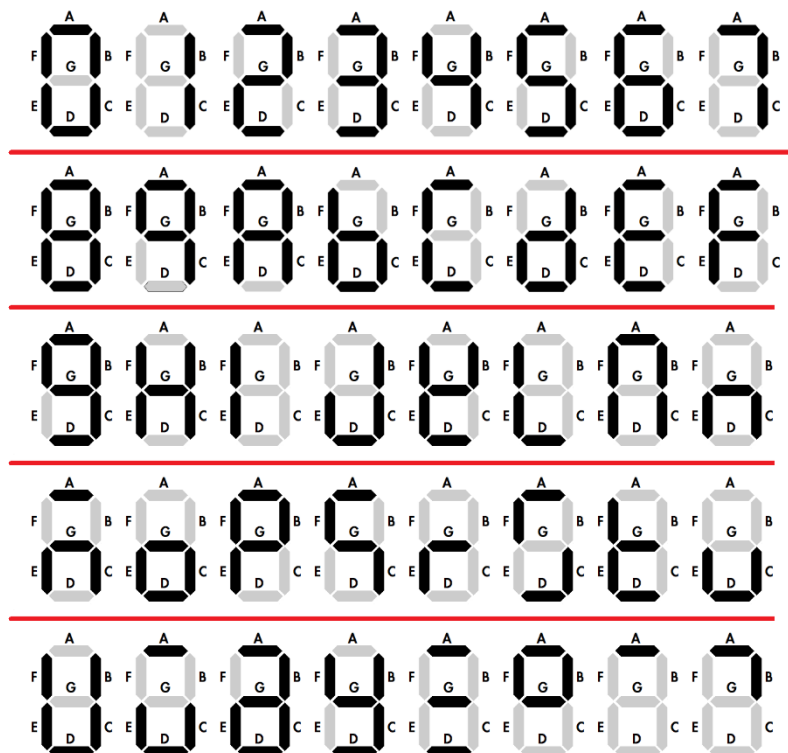
**Figura 23:** Lista de componentes en TINKERCAD.

62. Cierre el navegador web y todas las aplicaciones abiertas.
63. Apague la computadora, incluyendo el monitor.
64. Deje en orden su puesto de trabajo.

### EJERCICIOS.

Utilizando su cuenta en Tinkercad, realice la simulación para los ejercicios solicitados a continuación:

1. Utilice el circuito de la figura 13 y realice el código para mostrar los dígitos mostrados a continuación cada 2 segundos.



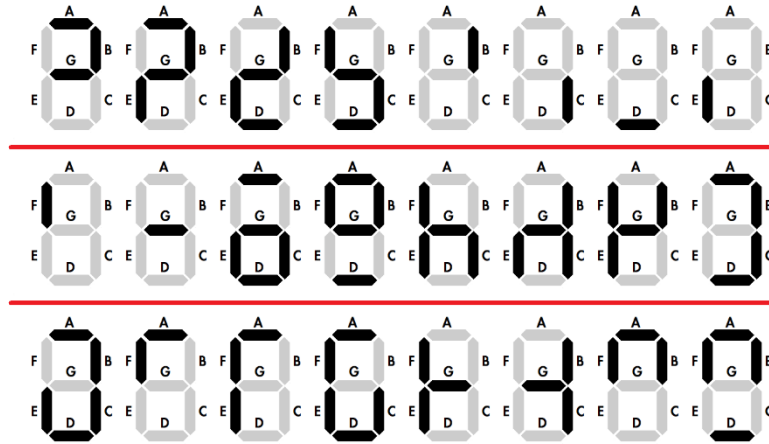


Figura 24: Valores a mostrar en el display de la Figura 13.

- Con la ayuda de la figura 25 y el código 5 (no funciona como se encuentra), genere un decodificador de 7 segmentos para 4 bits de entrada, debe cumplir con la tabla 3 cuando el código esté completo.

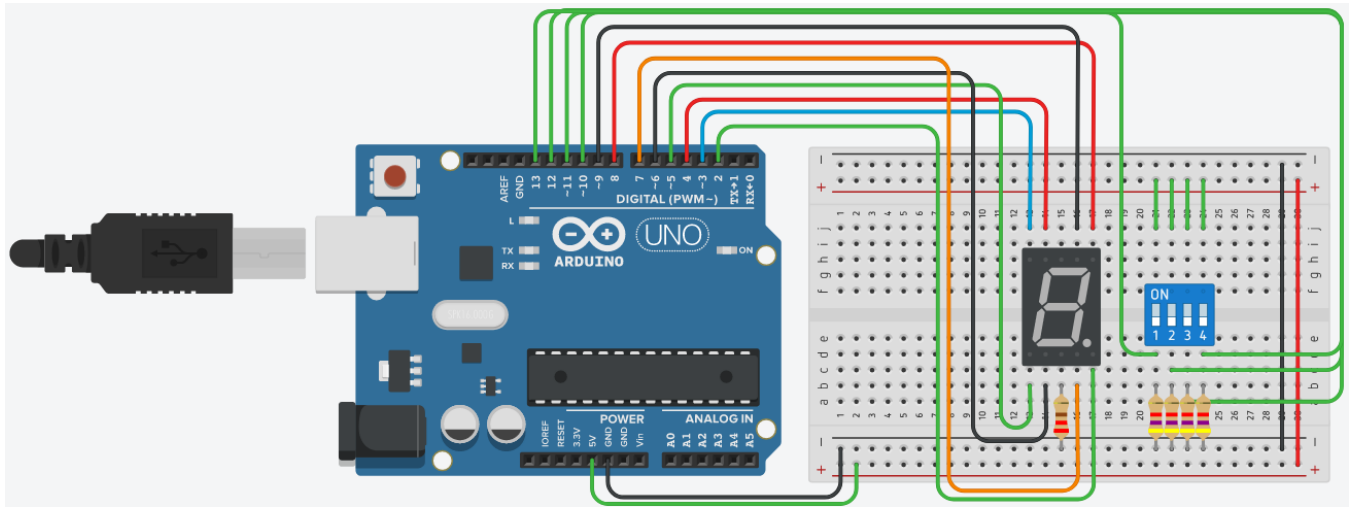


Figura 25: Control de entradas y salidas para realizar codificador binario a hexadecimal, display de **cátodo común**.

```
int valor = 0;
int Retardo = 100;
int entradas[4] = {13,12,11,10}; //Entradas de DIP switch
int salidas[8] = {9,8,7,6,5,4,3,2}; //Salidas a display

bool leds[4][8]{
    //Arreglo con estados de leds de display
    //{a,b,c,d,e,f,g,pd} Código de 7 segmentos en tinkercad
    {1,1,1,1,1,0,0}, //0
    {0,1,1,0,0,0,0}, //1
    {1,1,0,1,1,0,1}, //2
    {1,1,1,1,0,0,1}, //3
};

void setup() {
    for (int i = 0; i < 8; i++) { //Inicializa las salidas
```

```

        digitalWrite(salidas[i], LOW);
    }
}

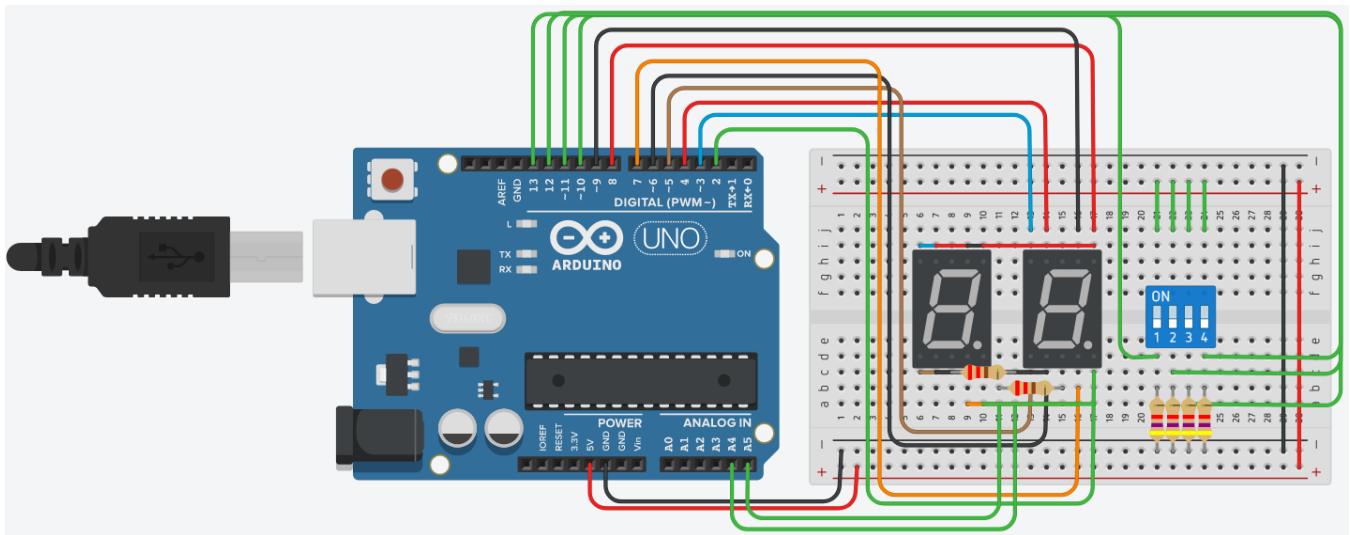
void loop() {
    //la variable valor representa un número de 0 a 15
    //tomado de la lectura de las entradas
    valor = digitalRead(entradas[0]);
    valor = valor + (digitalRead(entradas[1])*2);
    valor = valor + (digitalRead(entradas[2])*4);
    valor = valor + (digitalRead(entradas[3])*8);

    delay(Retardo);
}

```

**Código 5:** Parte del Sketch del decodificador de binario (4 bits) a siete segmentos.

3. Modifique el código anterior para ser un decodificador de binario (4 bits) a decimal (2 displays). Considere llevar conteo de decenas y unidades. Ver figura 26.

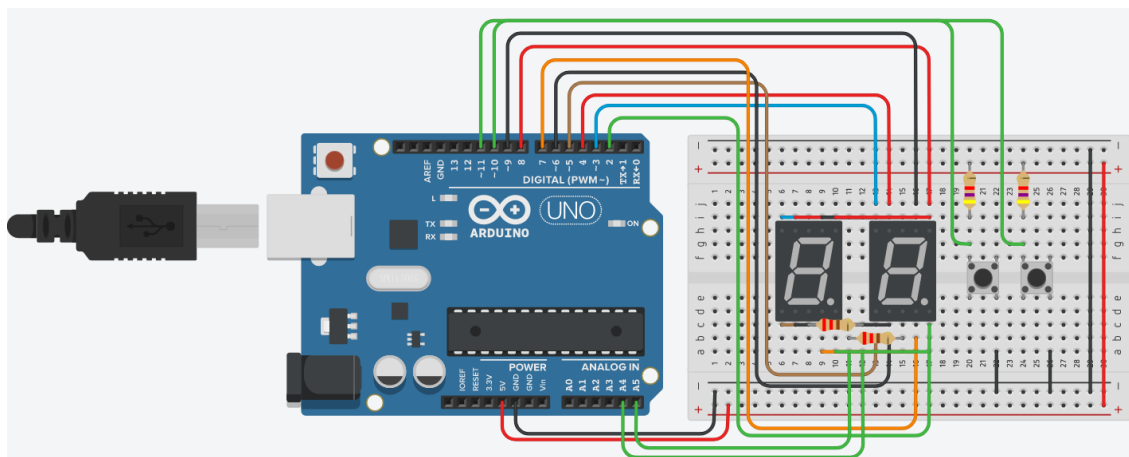


**Figura 26:** Circuito propuesto para el código 5.

4. Con la ayuda de la figura 27, desarrolle un código para generar un contador decimal ascendente/descendente de 00 a 99, el incremento se realiza al presionar el botón 1 y decremento al presionar el botón 2.

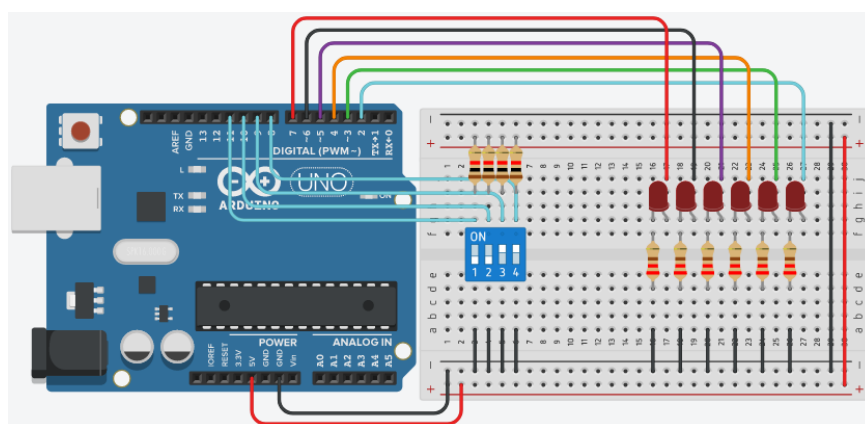
**Notas para tomar como válido la ejecución de este ejercicio:**

- Considere que debe utilizar **bucles if, for**, uso de **arreglos**.
- Se debe alternar la escritura de los displays para dar el efecto que ambos están encendidos al mismo tiempo, los manejadores de los displays son los pines 12 y 13.
- Considere llevar conteo de decenas y unidades.
- El código anti-rebote para los pulsadores.



**Figura 27:** Circuito propuesto para el código 5.

5. Modifique el código anterior para realizar un contador hexadecimal ascendente/descendente de 00 a FF, el incremento se realiza al presionar el botón 1 y decremento al presionar el botón 2.
6. Modifique el circuito de la figura 25 y código anterior para leer 7 bits de entrada, 3 displays de salida. Realice un decodificador de binario (7 bits) a decimal (000 a 127).
7. Modifique el código anterior para que sea binario (7 bits) a hexadecimal (00 a 7F).
8. Dada la figura 28, realice código y circuito (de 8 salidas y 4 entradas) que permita realizar diferentes secuencias de luces, dependiendo del valor colocado en el minidip celeste, así sea la secuencia seleccionada, investigue que son los registros de desplazamiento.



**Figura 28:** Circuito propuesto.

## BIBLIOGRAFÍA.

1. Páginas web:
  - <https://www.tinkercad.com>
  - <https://botscience.wordpress.com/2012/06/05/historia-de-arduino-y-su-nacimiento/>
  - <https://vimeo.com/18390711>
  - Sitio web de los desarrolladores: [www.arduino.cc](http://www.arduino.cc)
  - Referencia en línea del lenguaje: <http://arduino.cc/en/Reference/HomePage>